

Audio Genre Classification using Deep Learning on Synthetic Mashups

Miriyala Indu Vardhan(21F3000478)

April 3, 2026

Abstract

This project focuses on classifying audio samples into predefined genres using deep learning techniques. Unlike traditional datasets, the training data is synthetically generated by combining multiple audio stems along with environmental noise, making the task more realistic and challenging. The pipeline includes audio pre-processing, mashup generation, spectrogram conversion, and model training. Three models were implemented: a custom CNN, ResNet-18, and EfficientNet-B2. Among these, EfficientNet-B2 achieved the best performance due to its efficient scaling and strong feature extraction capabilities. After further enhancements to the data augmentation pipeline—including increased audio duration, tempo stretching, wider stem weighting, and stronger noise augmentation—the Kaggle submission score improved from 0.84 to 0.86396. The results highlight the effectiveness of transfer learning and synthetic data generation in improving robustness for audio classification tasks.

1 Introduction

1.1 Problem Statement

The goal of this project is to classify audio samples into predefined genres. The dataset is constructed using audio stems (drums, bass, vocals, others) which are combined to form synthetic mashups. Environmental noise is also incorporated to simulate real-world conditions.

1.2 Objective

The objective of this project is to generate realistic audio samples using mashup techniques, convert them into spectrogram representations, train deep learning models for classification, and evaluate their performance on unseen test data.

1.3 Synthetic Data Generation Pipeline

Instead of relying on fixed audio clips, each sample is generated on-the-fly using a combination of audio stems and environmental noise, significantly improving data diversity and model generalization.

1.3.1 Audio Standardization

All audio signals are standardized by resampling to 22.05 kHz, converting to mono, and fixing the duration to 10 seconds. Longer clips are randomly cropped, while shorter clips are zero-padded.

1.3.2 Mashup Generation

Each training sample is constructed by combining four stems (drums, bass, vocals, and others), each selected from different songs within the same genre. This prevents overfitting to specific tracks. Random weights are assigned to each stem:

$$w \sim \mathcal{U}(0.7, 1.3)$$

The final music signal is computed as:

$$x_{\text{music}} = \frac{w_d x_d + w_b x_b + w_v x_v + w_o x_o}{w_d + w_b + w_v + w_o}$$

1.3.3 Noise Injection

Environmental noise from ESC-50 is added by mixing two randomly selected noise clips:

$$x_{\text{final}} = x_{\text{music}} + \lambda x_{\text{noise}}, \quad \lambda \sim \mathcal{U}(0.05, 0.30)$$

1.3.4 Normalization

The final signal is normalized to prevent clipping:

$$x = \frac{x}{\max(|x|)}$$

1.3.5 Feature Extraction

Audio signals are converted into Mel-spectrograms using:

- FFT size: 2048
- Hop length: 512
- Mel bands: 128

The spectrogram is converted to log scale (dB) for improved representation.

2 Modeling & Experimentation

2.1 Model 1: Custom CNN

A baseline CNN was implemented with three convolutional layers (1→16→32→64), followed by max pooling, global average pooling, and a fully connected layer. The model learns hierarchical features from spectrogram inputs.

Optimization: Adam optimizer with learning rate 1×10^{-3} .

Metrics: Accuracy and weighted F1-score.

2.2 Model 2: Fine-Tuned ResNet-18

A pretrained ResNet-18 model was fine-tuned for the task. The first layer was modified for single-channel input, and the final layer was replaced to match the number of classes. All layers were trained to adapt features to the audio domain.

Optimization: Adam optimizer with learning rate 1×10^{-3} and weight decay 1×10^{-3} .

Loss: Cross-entropy loss.

Metrics: Accuracy and weighted F1-score.

2.3 Model 3: Fine-Tuned EfficientNet-B2

EfficientNet-B2 was used as the primary model. The first convolution layer was adapted to single-channel input by averaging pretrained RGB weights. The final classifier was modified for the target classes, and the entire network was fine-tuned.

Optimization: AdamW optimizer with learning rate 5×10^{-4} and weight decay 1×10^{-4} .

Loss: Cross-entropy loss.

Metrics: Accuracy and weighted F1-score.

3 Comparative Analysis

A comparative study was conducted across CNN, ResNet-18, and EfficientNet-B2 using different batch sizes.

Table 1: Performance comparison across models

Model	Batch	LR	Train Loss	Train Acc	Train F1	Val Loss	Val Acc	Val F1	Best F1
CNN	16	0.0005	0.6817	0.7618	0.7579	1.0484	0.665	0.7746	0.8035
CNN	32	0.0005	0.8549	0.7136	0.7068	1.1124	0.650	0.7710	0.7814
CNN	64	0.0005	1.0017	0.6586	0.6489	1.1022	0.642	0.7648	0.7648
ResNet-18	16	0.0001	0.0189	0.9960	0.9972	0.8657	0.755	0.8432	0.8645
ResNet-18	32	0.0001	0.0170	0.9964	0.9970	0.9069	0.755	0.8476	0.8595
ResNet-18	64	0.0001	0.0676	0.9790	0.9796	1.1234	0.708	0.8109	0.8341
EfficientNet-B2	16	0.0001	0.0217	0.9930	0.9946	0.8507	0.812	0.8873	0.8921
EfficientNet-B2	32	0.0001	0.0267	0.9924	0.9932	0.9823	0.800	0.8746	0.8939
EfficientNet-B2	64	0.0001	0.0182	0.9952	0.9955	0.9706	0.788	0.8673	0.8723

EfficientNet-B2 achieved the best performance, reaching 81.2% validation accuracy and the highest F1-score of 0.892. ResNet-18 significantly improved over the CNN due to transfer learning, while the CNN showed limited performance due to its shallow architecture.

Batch size influenced performance across all models, with smaller batch sizes generally yielding better generalization. Both ResNet and EfficientNet exhibited high training accuracy ($> 99\%$), with a slight gap between training and validation metrics indicating mild overfitting. EfficientNet-B2 maintained the best balance between training and validation performance.

The best configuration was EfficientNet-B2 with batch size 16 and learning rate 1×10^{-4} .

4 Improvements to Boost Accuracy

To further improve classification performance beyond the initial Kaggle score of 0.84, several enhancements were made to the data augmentation pipeline and training setup. These changes collectively raised the submission score to **0.86396**.

4.1 Increased Audio Duration (10s $\rightarrow$ 15s)

The audio clip duration was increased from 10 seconds to 15 seconds. Longer clips provide the model with more temporal context per sample, allowing it to capture a richer set of rhythmic patterns, transitions, and genre-specific structures within each spectrogram. This directly results in larger Mel-spectrogram inputs with more time frames, giving the convolutional layers additional information to learn from.

4.2 Tempo Stretching via PyRubberband

A random tempo stretch was introduced during stem loading using the PyRubberband library. Each audio stem is time-stretched by a factor sampled uniformly:

$$r_{\text{stretch}} \sim \mathcal{U}(0.8, 1.2)$$

This $\pm 20\%$ tempo variation simulates the tempo misalignment present in test-time mashups, where stems from different songs may not share the same BPM. By exposing the model to tempo-varied inputs during training, it becomes more robust to such mismatches during inference.

4.3 Wider Stem Weight Range

The random weight range for mixing individual stems was widened from $\mathcal{U}(0.7, 1.3)$ to $\mathcal{U}(0.5, 1.5)$:

$$w \sim \mathcal{U}(0.5, 1.5)$$

This greater variation in stem volumes forces the model to handle a wider dynamic range of instrument balances. For example, some mashups may now be heavily drum-dominated while others are vocal-heavy, preventing the model from relying on a fixed volume profile for classification.

4.4 Enhanced Noise Augmentation

The noise injection strategy was strengthened in two ways:

Three-clip noise mixing: Instead of mixing two noise clips from ESC-50, three noise clips are now selected and cross-mixed to produce a 15-second noise signal. Specifically, three clips (n_1, n_2, n_3) are combined pairwise with a random mixing ratio $\alpha \sim \mathcal{U}(0.3, 0.7)$:

$$m_1 = \alpha \cdot n_1 + (1 - \alpha) \cdot n_2, \quad m_2 = \alpha \cdot n_2 + (1 - \alpha) \cdot n_3, \quad m_3 = \alpha \cdot n_3 + (1 - \alpha) \cdot n_1$$

These three segments are concatenated to form the full 15-second noise track, producing more diverse and complex background noise patterns.

Wider noise level range: The noise mixing coefficient was expanded from $\lambda \sim \mathcal{U}(0.05, 0.30)$ to $\lambda \sim \mathcal{U}(0.05, 0.40)$, allowing some training samples to contain substantially more noise. This makes the model more resilient to noisy conditions at test time.

4.5 Impact on Performance

These augmentation improvements collectively increased the Kaggle submission score from **0.84 to 0.86396**, representing a meaningful gain. The key insight is that better alignment between training-time augmentation and test-time conditions—particularly tempo variation and noise levels—directly translates to improved generalization on unseen mashups.

5 Conclusion

This project demonstrates that synthetic data generation combined with deep learning is effective for audio classification tasks. The use of spectrogram representations enables CNN-based models to extract meaningful features from audio signals. Transfer learning significantly improves performance, as seen with ResNet-18 and EfficientNet-B2.

EfficientNet-B2 achieved the best results due to its efficient scaling and superior feature extraction capability. The subsequent improvements to the data augmentation pipeline—including longer audio duration, tempo stretching, wider stem weighting, and enhanced noise injection—further boosted the Kaggle score from 0.84 to 0.86396, underscoring the importance of training-time augmentation that mirrors test-time conditions. However, some overfitting was observed, suggesting scope for further improvements.

6 Future Work

Future improvements include exploring transformer-based models such as audio spectrogram transformers, applying advanced data augmentation techniques, and performing hyperparameter tuning to further improve performance.